

Embedded Systems Homework #1

Nidhi Borhade
StudentId : 109998404

September 2023

Contents

1	Difference Monolithic OS, Layered OS and Microkernel OS	3
1.1	Monolithic OS	3
1.2	Layered OS	4
1.3	Microkernel OS	4
1.4	Differences	5
2	Real-time Embedded System	6
2.1	Hard Real Time Embedded System	6
2.2	Soft Real Time Embedded System	7
3	Types of Embedded Processors	8
3.1	General Purpose Processors	8
3.2	Microprocessors	8
3.3	Mircococontrollers	8
3.4	Digital Signal Processors	8
3.5	Single Purpose Embedded Processors	8
3.6	System-on-Chip	9
4	Embedded Operating Systems	10
4.1	Hybridkernel OS	10
4.2	Nanokernel OS	10
4.3	Exokernel OS	11
4.4	Monolithic OS	11
4.5	Layered Operating System	11

4.6	Microkernel OS	11
-----	--------------------------	----

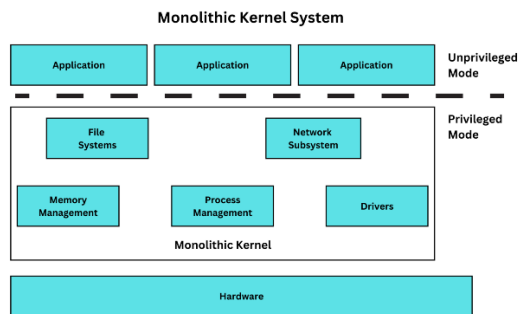
1 Difference Monolithic OS, Layered OS and Microkernel OS

1.1 Monolithic OS

A Monolithic Operating System is a straightforward operating system where the kernel directly manages file management, device management, process management, and memory management. It runs with a monolithic kernel, this Kernel is only one big program, which leads to the main problem of this OS.

If the Kernel crashes the whole system will crash. And if you consider that the Kernel is one big file, you will realise, it is hard to add new functionalities into the Kernel itself.

This type of Operating system is outdated and not really used anymore.



Source: <https://www.tutorialspoint.com/difference-between-monolithic-operating-system-and-layered-operating-system/>

tutorialspoint.com/difference-between-monolithic-operating-system-and-layered-operating-system/

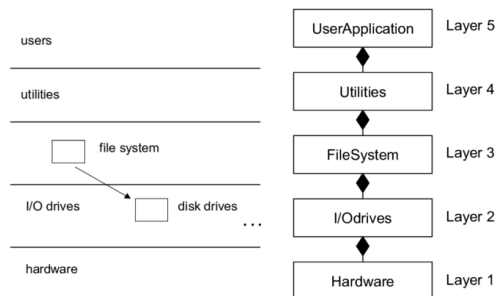
1.2 Layered OS

Layered Operating Systems are an alternative to Monolithic Operating Systems. All the different functions of the Operating System are separated into different layers, every layer has its own specific function. The Layering makes the maintaining of the system way easier.

All the layers communicate over well-defined interfaces, ensuring a structured and organized interaction.

If you think about the different layers, you will realise there is no single point of failure anymore, because all the layers are their own programs. (compared to monolithic OS). The layered OS is way more stable.

The performance can suffer due a lot of communication between the different layers. (there is overhead passing data and latency)



Source: [https://www.](https://www.researchgate.net/figure/An-example-of-the-use-of-a-Layered-OS-architecture_fig4_268397726)

[researchgate.net/figure/An-example-of-the-use-of-a-Layered-OS-architecture_fig4_268397726](https://www.researchgate.net/figure/An-example-of-the-use-of-a-Layered-OS-architecture_fig4_268397726)

1.3 Microkernel OS

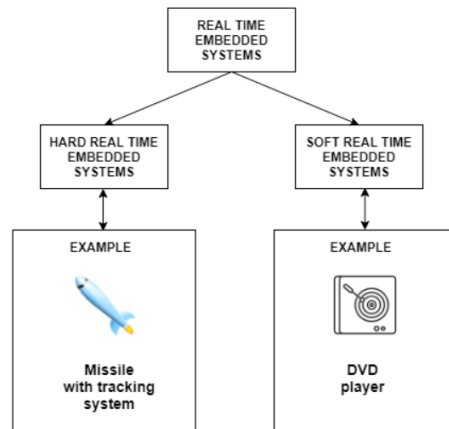
The Microkernel OS uses a microkernel, which is a kernel that provides a minimal amount of process and memory management services. User and Kernel services are implemented in different address spaces. If any User Service Fails, the OS is not affected, because the kernel is separated from it. It is easy to extend, new services are added to user address space. They are used most often in distributed systems, because features of the OS can be outsourced or changed.

1.4 Differences

Basis of Comparison	Monolithic OS	Layered OS	Microkernel OS
Basic	user services and kernel services are in same address space	organized into distinct layers for specific function	User and kernel services in different address spaces
Size	large	small	small
Performance	really fast	slower due layer communication	slower due communication
Security	if service crashes whole system crashes	no effect on kernel	no effect on kernel
Amount of Code Maintenance	less code hard to maintain, we have one big code block	more code complicated, hard to maintain	more code easy to maintain
Extendible	really hard to extend	easy to extend	easy to extend

2 Real-time Embedded System

Those systems that work with strict time constraints. When there is an embedded component in a real time system, it is known as a real time embedded system. It uses some special algorithm to make sure the critical process will finish in time. There are two different types of real time embedded systems :



Source: <https://www.tutorialspoint.com/Real-Time-Embedded-Systems#:~:text=Real%20time%20systems%20are%20those,a%20real%20time%20embedded%20system.>

2.1 Hard Real Time Embedded System

All the critical processes need to be finished within given time frame. If not finishing the process in given time can have serious consequences. Usually used in :

1. nuclear power plant
2. airplanes
3. medical system

2.2 Soft Real Time Embedded System

Trys to do the processes in real time, but if real time is not possible, its not crucial compared to Hard Real Time. Examples:

1. Multi Media Streaming Provider
2. Computer Games
3. Board Computer in a vehicle

3 Types of Embedded Processors

3.1 General Purpose Processors

General Purpose Processors are also known as CPU (central processing unit). These processors are capable of executing wide range tasks. Some CPU's even provide multi-coreing, which makes it possible to perform multiple tasks concurrently. Usually they are more expensive than other Embedded Processors.

3.2 Microprocessors

A Microprocessor is contained on a microchip, it is a CPU that executed the instructions from a computer programm.

3.3 Mircocontrollers

A microcontroller is a small, low-power computer that is contained in a single intregrated circuit (IC). It has on chip peripherals (timer, ports, analog-to-digital converters) & memory. The cost of Mircocontrollers is low, this is why they are used in daily consumer electronics.

3.4 Digital Signal Processors

Digital Signal Processors are special microprocessors that are designed to process analog signals and convert them into digital signals, which the computer can interpret. Usually they work in high speed, they can multi process, they provide common operations like Fast Fourier Transformation, and all that with really low power supply.

3.5 Single Purpose Embedded Processors

Single Purpose Embedded Processors are also known under the name application-specific embedded processors. They are designed to do one specific task or a certain set of tasks. Usually they have low-power consumption, a compact size, high reliability and minimum cost compared to general purpose Processors.

3.6 System-on-Chip

System-on-Chip (SoC) is an integrated circuit, that contains all components of a computer on a single chip. They are also known as power-efficient, high-performant, customizable (even component parts).

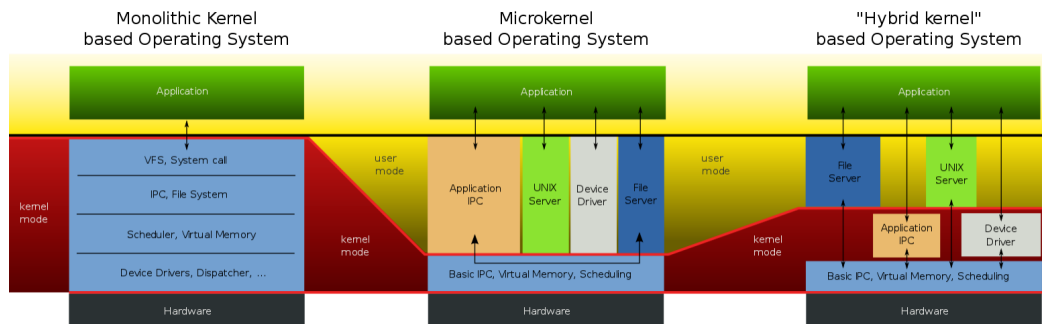
4 Embedded Operating Systems

4.1 Hybridkernel OS

The Hybridkernel OS is an architecture that attempts to combine features and benefits of microkernel & monolithic OS. In contrast to a microkernel, all (or nearly all) operating system services in a hybrid kernel are still in kernel space. There are none of the reliability benefits of having services in user space, as with a microkernel. However, just as with an ordinary monolithic kernel, there is none of the performance overhead for message passing and context switching between kernel and user mode that normally comes with a microkernel.

Examples that use the Windows NT Kernel:

1. Windows 11
2. Windows Server 2022
3. Xbox One



Source:

https://en.wikipedia.org/wiki/Hybrid_kernel#:~:text=A%20hybrid%20kernel%20is%20an,architectures%20used%20in%20operating%20systems.

4.2 Nanokernel OS

A nanokernel is a small kernel that offers hardware abstraction, but without system services. Larger kernels are designed to offer more features and manage more hardware abstraction. Modern microkernels lack system services as well, hence, the terms microkernel and nanokernel have become analogous.

4.3 Exokernel OS

The Exokernel OS is a vertical structured OS. The Kernel in this OS is basically just checking if there are resource conflicts, such as memory management.

4.4 Monolithic OS

[click to get to Monolithic section](#)

4.5 Layered Operating System

[click to get to Layered Operating System section](#)

4.6 Microkernel OS

[click to get to Micro Kernel section](#)